

Gradient boosted trees for UK All-Weather Flat handicaps

Paper 3 in a series on UK All-Weather racing outcomes

Patrick Gillen

Papers 2a and 2b changed the feature set and then the prediction objective for a conditional-logit model of UK All-Weather Flat handicaps. This paper holds both fixed and changes the function class instead, replacing the linear score with a gradient-boosted ensemble of decision trees fitted on the same depth-three Plackett–Luce likelihood, selected from a 72-point cross-validated grid on 5,022 training races. Scored on 2,183 held-out test races, the tree ensemble ranks no better than paper 2b’s linear predictor: a paired bootstrap of the two models’ P1_rank, Brier_place and pseudo- R^2 contains zero on every metric, despite the two being materially different models (within-race score correlation 0.918; a different top pick in 33% of races). Naive win-picking and single-bet betting value show the same pattern against both papers 2a and 2b. The market’s own probabilities beat the tree ensemble decisively on both ranking metrics — the one contrast in the paper whose interval excludes zero — continuing the pattern of every paper in this series. Changing the function class does not close the gap the feature set and the objective also left open.

Table of contents

1	Introduction	2
2	Data and features	3
2.1	The test universe	3
2.2	Feature set	3
2.3	Going affinity	5
3	Method	6
4	Model fitted	7
4.1	The objective	7

4.2	Tuning procedure	8
5	Results	9
5.1	Q1 — Ranking	9
5.2	Predictor importance	9
5.3	Q2 — Win-picking	12
5.4	Q3 — Betting value	12
6	Discussion	13
7	Appendix A: Derivation	15
8	Appendix B: Software and reproducibility	17
	Acknowledgments	17
	References	18

1 Introduction

Paper 2a extended paper 1’s feature set; paper 2b extended the objective from predicting the winner to predicting the finishing order. This paper holds both fixed and changes the function class instead: the linear score of a conditional logit is replaced by an ensemble of decision trees, fitted by gradient boosting on the same depth-three ranking likelihood paper 2b uses. A tree finds interactions among features on its own, rather than needing each one specified as a term in advance, and it is the model class chosen specifically to admit going affinity — a horse’s own record on the going it faces today — without the explicit interaction a linear model would need and paper 2b did without.

The three questions are the ones papers 2a and 2b ask, and the answer to each is the same. **Q1 — ranking performance.** Does the tree ensemble predict the finishing order better than paper 2b’s linear model? It does not: a paired bootstrap of the two models’ ranking metrics on the same test races contains zero on every metric compared, and the market’s own probabilities beat both. **Q2 — win-picking.** Does richer, data-found interaction structure make it a better winner-picker than papers 2a or 2b? It does not: on the naive betting rule the point estimates run the wrong way against both comparators, and the intervals do not distinguish any of the three models. **Q3 — betting value.** Turned into a single bet per race, does that reach a return paper 2b’s own single-bet rule does not already reach? It does not: win, place and each-way returns overlap paper 2b’s figures throughout.

The tree ensemble and the linear predictor are not the same model reaching the same answer by default — they disagree on the top horse in a third of test races — but they rank, pick and

bet alike, and both lose to the market by about the margin the series has shown throughout. What that tie means, and what it does not settle, is taken up in Section 6.

Section 2 sets out the shared data and feature set, including the going-affinity feature this change of model class was chosen to accommodate; Section 3 motivates the objective and the model class; Section 4 describes the fit and how it was tuned; Section 5 answers the three questions in order; and Section 6 draws the series’ three changes together.

2 Data and features

The data, the race-selection rules, and the feature set before the additions below are identical to papers 2a and 2b; we refer the reader there (paper 2a, §2; paper 2b, Section 2) rather than repeat them. The chronological race-level train/test split at 2012-12-30 and the complete-case rule (a race is dropped if any modelling feature is undefined for any of its runners) both carry over unchanged.

2.1 The test universe

A win backtest needs only to know who won, so paper 2b’s largest test universe keeps every race with a usable win probability. The ranking construction instead needs a clean top three — finishing positions one, two and three each recorded exactly once — so it works from a smaller universe. Paper 3 shares that second requirement, so it uses the 2,183-race universe throughout, and Section 5 reports every comparison against papers 2a and 2b on both universes.

2.2 Feature set

Table 1 lists the 24 features the model is fitted on: paper 2b’s own final specification, plus the four going-affinity features introduced in this paper.

Table 1: Paper-3 feature set: 24 features (course and the sex indicators are each one row here but several terms).

Feature	Description	Type	Source
pos_lag1_nonzero	Most recent prior finishing position (1st-4th; 0 if none)	horse	paper 1
pos_lag2_nonzero	Second most recent prior finishing position	horse	paper 1
age_diff	Symmetric distance from peak age (3 on AW)	horse	paper 1
days_LTO_log	Log days since last run	horse	paper 1
trainerSR	Cumulative pre-race trainer strike rate	horse	paper 1
sireSR	Cumulative pre-race sire strike rate	horse	paper 1

Table 1: Paper-3 feature set: 24 features (course and the sex indicators are each one row here but several terms).

Feature	Description	Type	Source
jockeySR	Cumulative pre-race jockey strike rate	horse	paper 2a
entire / gelding	Sex indicators	horse	paper 1
cheekpieces	Tack indicator	horse	paper 1
rel_weight	Weight carried minus race-mean weight	horse	paper 2a
or_relative	Official rating minus race-mean rating (0 where undefined)	horse	paper 2a
or_missing	Indicator: official rating was undefined	horse	paper 2a
trainer_aw_pre-mium	Trainer AW strike rate minus overall strike rate	horse	paper 2a
has_wins	Indicator: any prior career win	horse	paper 2a
stall_normalised	Starting stall divided by field size	horse	paper 2a
course (4 indicators)	Which of the four AW courses	race	paper 2a
going_runs_prior	Count of prior runs on today’s kind of going	horse	paper 3
going_sr_shrunk	Win rate on today’s kind of going, shrunk toward career rate	horse	paper 3
going_sr_delta	going_sr_shrunk minus overall career win rate	horse	paper 3
going_ordinal	Today’s going, on a common fast-to-slow scale	race	paper 3

Two substitutions replace terms that exist only to work around what a conditional logit needs and a tree does not.

- **Position lags.** Paper 2b encodes each prior finishing position as a pair of terms — an indicator for “no prior run” and the raw position where one exists — because a linear model needs the split to avoid reading a real finish on the same numeric scale as “no run.” A tree splits on the raw position directly, so the indicator is dropped: four terms become two.
- **Course and draw.** Paper 2b’s course effect only appears crossed with the horse-level draw, because a conditional logit cannot use a race-level covariate on its own — it cancels in the softmax. Here the draw and the four course indicators are entered separately, and a tree is free to split on either, or both together, at any depth, for any course — not only the ones the linear reduction happened to keep. Three terms become five.

Two further cases keep paper 2b’s own form. `or_relative` and its companion flag `or_missing` stay as built: the imputation is a considered missing-not-at-random strategy in its own right, not merely an encoding workaround. `days_LTO_log` also carries over unchanged, since a tree only asks whether a value falls above or below a threshold, and taking logs does not change which runners fall on which side.

2.3 Going affinity

`going_runs_prior`, `going_sr_shrunk`, `going_sr_delta` and `going_ordinal` are the one feature paper 2 deferred to this paper — going is a race-level quantity that cancels in a conditional-logit softmax unless interacted with a horse-level term, and a tree needs no such interaction. They are built from each horse’s full cross-surface career history, strictly before today’s race.

Today’s going is 98.9% “Standard” across the 5,022 training races (`going_ordinal` = 4 on the project’s 1–7 fast-to-slow scale). Because “Standard” pools AW Standard with turf Good — the two buckets are defined within surface, at that surface’s own terciles of going, and AW’s terciles both land on Standard — `going_sr_shrunk` and `going_sr_delta` describe, for nearly every runner, a horse’s win rate on Standard-or-Good ground relative to its own career rate. That is a measure of ground-type form, not a preference for today’s specific going in any finer sense; with going this close to constant across the training data, there is little room for a finer sense to be estimated from it.

A horse with very few prior runs on today’s kind of going has an unreliable win rate on it, so the figure used, `going_sr_shrunk`, is a blend of that rate and the horse’s overall career rate — weighted toward the career rate when the horse has run few times on this going, and toward the observed rate as those runs accumulate: $(\text{wins}_{\text{bucket}} + m \cdot \text{career_sr}) / (\text{runs}_{\text{bucket}} + m)$, with $m = 5$ a judgement call, not derived from the data or tuned. `going_sr_delta` is that shrunk rate minus the career rate, the term that separates going affinity from general ability. `going_sr_delta` is exactly zero by construction whenever every one of a horse’s prior runs falls in the same going bucket as today’s (24.0% of the 45,555 training runner-rows with a defined value), and undefined for a horse with no prior run in today’s bucket, including a true debut (0.90% of training runner-rows, 0.64% of test). These values are left missing rather than imputed: the model learns a default routing direction for a missing value at each split, rather than being given a stand-in value in its place.

Table 2: Training-split win rate by `going_sr_delta` decile, with mean field size.

Decile	Runner-rows	Delta range	Win rate	Mean field size
1	4,746	[-0.158, -0.029]	10.7%	9.9
2	4,746	[-0.029, -0.006]	8.7%	10.1
3	4,746	[-0.006, 0.000]	10.1%	10.0
4	4,746	[0.000, 0.000]	12.2%	9.4
5	4,745	[0.000, 0.004]	11.2%	9.9

Table 2: Training-split win rate by going_sr_delta decile, with mean field size.

Decile	Runner-rows	Delta range	Win rate	Mean field size
6	4,745	[0.004, 0.009]	9.9%	10.1
7	4,745	[0.009, 0.014]	10.5%	10.0
8	4,745	[0.014, 0.021]	10.6%	10.1
9	4,745	[0.021, 0.033]	11.3%	10.0
10	4,745	[0.033, 0.204]	13.7%	9.8

The win rate rises from decile 1 to decile 10 without a corresponding trend in mean field size — the ten deciles’ mean field sizes range from 9.4 to 10.1 runners, which rules out a smaller or larger field as the mechanical explanation. A race-clustered bootstrap of the win-rate difference between decile 10 and decile 1 — the same resample-races-with-replacement design as the ROI-difference bootstrap used throughout this series, races resampled 2000 times with seed 42, the decile assignment and both win rates recomputed fresh within each resample rather than treating the two deciles as independent samples — gives a point estimate of +0.030 with a 90% CI of [0.019, 0.040].

3 Method

Every loss an ensemble of trees is ordinarily fitted against is a sum of one term per observation: the correction owed to a given observation depends on that observation’s own error and nothing else. Racing does not fit this pattern. A finishing order ranks an entire field against itself — a horse’s chance is only ever defined relative to the others it runs against on the day — so the natural unit is not the horse but the race, and a single term of the loss has to span every runner in the field at once. This is the one structural change this paper makes to the method, and everything else below follows from it.

The likelihood is the one built for paper 2b. Each horse in a race is given a score by the model, and within that race the scores are turned into win probabilities by a softmax: a horse’s probability is its own score, exponentiated, as a share of the same sum taken over every runner still in the field. The finishing order becomes a likelihood by working down through the placings — first asking which horse won, out of the whole field; then which horse came second, with the winner now removed; and so on to the top three, as paper 2b uses — and combining the probability of each step. This is the Plackett–Luce likelihood, and it is what the ensemble is trained to maximise (Section 7). Stopping after the first step, and modelling only the winner, recovers the conditional logit of paper 1; the likelihood used here is that same construction, carried further into an ordering of the whole field, as paper 2b already established.

A score is not a prediction of anything a race result reveals directly, the way a fitted value in an ordinary regression stands in for an observable response. It exists only to be compared,

through the softmax, against the other scores in its own race. What the model observes is which horse won, which came second, and so on, and it adjusts scores so that the probabilities those results imply move toward what happened: a horse the current scores under-favour has its score raised, one they over-favour has it lowered, and the outcomes of many past races determine, indirectly, what score a horse with a given set of characteristics ends up receiving. Ability, in other words, is inferred from results rather than measured directly.

A tree does not need to be told in advance which characteristics interact. A branch that splits first on which course a race is run at, and only then, further down, on the horse’s stall position, produces a value that applies only to horses satisfying both conditions — a particular draw, at a particular course. The effect of the draw is then free to differ from one course to the next, which is what an interaction means; a linear predictor can only express this by being handed that interaction term explicitly, and only for the combinations a modeller thinks to specify in advance. A tree of depth d can express interactions among up to d variables this way, discovered from the data rather than listed beforehand — an advantage wherever a horse’s characteristics are unlikely to combine additively, which every conditional-logit model in this series has had to assume regardless.

Put together, the space the model works over is partitioned into regions on which a horse’s score is constant, and because every split is a threshold on one variable at a time, those regions are rectangles aligned to the feature axes: a horse shares a region with another only if it matches it on every threshold the ensemble has found, whatever else distinguishes them. Two horses in the same race will generally fall in different regions, and it is that difference, carried through the softmax, that becomes the model’s ordering of the field.

4 Model fitted

4.1 The objective

The GBT is fitted under the same objective as paper 2b’s ranking model: the depth- $k = 3$ Plackett–Luce likelihood over each race’s finishing order — the derivation is identical, only the function class changes (from a linear predictor to an additive tree ensemble). This objective is supplied as a gradient and a Hessian, one value of each per training row (Section 7). The Hessian is the diagonal only — the second derivative of each row’s own score, dropping the cross terms between runners in the same race that the full likelihood actually has. This is a standard approximation for a grouped loss fitted one row-level Hessian at a time, and the same one the function class’s own built-in ranking objectives make.

Those built-in ranking objectives (pairwise and list-wise losses) are a different objective from the Plackett–Luce likelihood and were not used; the custom objective is what keeps this paper’s model and paper 2b’s comparable at the same likelihood.

4.2 Tuning procedure

The tuning grid was fixed in advance of fitting anything, before any test-set number existed: a full factorial over five hyperparameters, 72 points in all.

Table 3: The 72-point tuning grid (full factorial).

Hyperparameter	Values tested
max_depth	2, 3, 4, 6
eta	0.01, 0.03, 0.1
min_child_weight	1, 5, 20
subsample	0.7, 1
colsample_bytree	0.7

Each point was fit under race-grouped 5-fold cross-validation — races, not runner-rows, are assigned to folds, so no race is split across an analysis/assessment pair — with early stopping determining the number of boosting rounds from each fold’s own held-out score (Section 8 has the implementation budget behind this grid).

The winning point is the one with the highest fold-mean score; among points within 0.001 of that maximum, the rule prefers the shallowest tree, then the fewest rounds. The selected configuration is max_depth 3, eta 0.03, min_child_weight 1, subsample 0.7, colsample_bytree 0.7.

It is refit once on all training data, with no held-out set left to early-stop against: the number of rounds is the mean, across its five folds, of each fold’s own early-stopped count, rounded — 698 rounds. The resulting fit’s training pseudo- R^2 is 0.09200, in-sample.

The pre-declared tie-break window was wider than the paired standard errors observed near the top of the grid, so the rule selected among points a paired test would not call tied. A refit at the grid maximum, taken at its own fold-mean rounds, produces materially the same model — 8 of the top ten features by gain importance hold the identical rank in both fits.

The selected configuration is not on a boundary of the grid (neither `max_depth` nor `eta` is the largest value tested).

Row subsampling operates at the level of individual horses, not races, because there is no race-level grouping to subsample by. Under a grouped likelihood the runners in a race are not independent, so the subsampling unit and the likelihood’s own grouping unit do not match — a limitation Section 6 returns to.

5 Results

The three questions are the same ones papers 2a and 2b ask, in the same order: how well the model ranks, how well it picks winners, and whether that translates into betting value. All three are scored on the 2,183-race test universe (Section 2).

5.1 Q1 — Ranking

Table 4: Ranking metrics on the 2,183 test races: the GBT, paper 2b’s exploded model, paper 2a’s win model, and the discounted-Harville market. *Paper 2a’s figures are computed fresh from its stored test predictions for this comparison and were not part of its published results.

Metric	GBT	Paper 2b	Paper 2a*	Market
P1_rank (higher is better)	0.00401	0.00402	0.00396	0.00543
Brier_place (lower is better)	0.20100	0.20100	0.20200	0.18800

On the same test races, test-split McFadden pseudo- R^2 is 0.05145 for the GBT and 0.05125 for paper 2b.

A paired race-level bootstrap ($B = 2000$, seed 42) of the GBT against paper 2b gives a 90% interval containing zero on all three metrics: P1_rank -5.68e-06 [-4.91e-05, 3.55e-05], Brier_place +0.0001 [-0.0005, 0.0008], and test pseudo- R^2 -0.0002 [-0.0021, 0.0015]. The same bootstrap against the discounted-Harville market excludes zero on both metrics: P1_rank -1.41e-03 [-1.55e-03, -1.28e-03], Brier_place +0.0139 [0.0126, 0.0153].

The tree ensemble and the linear predictor rank equally well on this test set: neither ranking metric distinguishes them. The market ranks better than both, on both metrics, and this is the only contrast in the paper whose interval excludes zero.

That the two models tie is not the same as the two models agreeing race by race. Within-race-centred scores from the GBT and paper 2b correlate at $r = 0.918$ (Spearman 0.923), and the two models pick a different top horse in 32.7% of races. They are different models that rank equally well.

5.2 Predictor importance

Gain importance is an in-sample accounting of where the fitted loss went — evidence the model used a feature, not evidence it helps out of sample.

Table 5: Gain importance, all 24 features, in-sample.

Feature	Rank	Gain
or_relative	1	0.2361
pos_lag1_nonzero	2	0.1773
rel_weight	3	0.0879
trainerSR	4	0.0793
days_LTO_log	5	0.0789
jockeySR	6	0.0574
pos_lag2_nonzero	7	0.0442
trainer_aw_premium	8	0.0422
going_sr_shrunk	9	0.0338
sireSR	10	0.0298
going_sr_delta	11	0.0240
going_runs_prior	12	0.0223
age_diff	13	0.0214
stall_normalised	14	0.0189
entire	15	0.0124
or_missing	16	0.0098
gelding	17	0.0092
cheekpieces	18	0.0040
has_wins	19	0.0024
course_Southwell	20	0.0023
course_Wolverhampton	21	0.0020
course_Kempton	22	0.0018
going_ordinal	23	0.0014
course_Lingfield	24	0.0012

Permutation importance, scored on the test split, is the out-of-sample measure, reported in two variants. Shuffling which horse holds a value within its own race is the right test for a horse-level feature, but it is a structural no-op for a feature constant within every race — today’s going, which course the race is at — so those five race-level features are instead permuted across races.

Table 6: Within-race permutation importance, test split, 19 horse-level features, 30 repeats.

Feature	Rank	Mean drop	SD
or_relative	1	0.05457	0.00234
pos_lag1_nonzero	2	0.02236	0.00098
rel_weight	3	0.02125	0.00126
days_LTO_log	4	0.00906	0.00082

Table 6: Within-race permutation importance, test split, 19 horse-level features, 30 repeats.

Feature	Rank	Mean drop	SD
trainerSR	5	0.00793	0.00069
jockeySR	6	0.00537	0.00060
pos_lag2_nonzero	7	0.00506	0.00061
age_diff	8	0.00242	0.00033
gelding	9	0.00208	0.00040
entire	10	0.00181	0.00041
going_sr_shrunk	11	0.00172	0.00027
trainer_aw_premium	12	0.00136	0.00034
going_runs_prior	13	0.00116	0.00026
going_sr_delta	14	0.00101	0.00026
sireSR	15	0.00068	0.00036
or_missing	16	0.00030	0.00005
stall_normalised	17	0.00018	0.00021
cheekpieces	18	0.00015	0.00022
has_wins	19	0.00014	0.00005

Table 7: Across-race permutation importance, test split, 5 race-level features, 30 repeats.

Feature	Rank	Mean drop	SD
course_Southwell	1	8.5e-05	3.3e-05
course_Lingfield	2	1.9e-05	1.4e-05
going_ordinal	3	-1.1e-05	9.0e-06
course_Kempton	4	-1.3e-05	2.1e-05
course_Wolverhampton	5	-3.9e-05	3.6e-05

The four course indicators are split on between 19 and 23 times each in the fitted trees — evidence the model uses them, since within-race permutation still returns exactly zero for every one.

Permutation also understates a correlated pair of predictors, since permuting one leaves its partner free to compensate for the lost signal — `going_sr_shrunk` and `going_sr_delta` correlate at $r = 0.35$ by construction, so their individual ranks in Table 6 are a lower bound on their joint contribution, not a point estimate.

The three horse-level going features sit mid-table, each clearing its own noise floor: `going_sr_shrunk` ranks 11, `going_runs_prior` 13, and `going_sr_delta` 14 of 19. `going_ordinal` ranks last of the five race-level features, its mean drop smaller in magnitude than its own standard deviation — indistinguishable from noise, as 98.9% of training races

sharing one going value predicts. A model carrying these four features ranks no better, out of sample, than paper 2b’s model, which did not have them.

5.3 Q2 — Win-picking

Owen’s naive rule (model win probability above 0.15, model/market ratio above 1.3), priced at real starting prices.

Table 8: Naive win backtest (prob > 0.15, ratio > 1.3), real starting prices. Published columns are each model’s own reported universe; restricted recomputes the same stored predictions on the GBT’s 2,183-race universe, no refit.

Model	Universe	Bets	Wins	ROI
GBT	2,183	1,424	154	-21.3%
Paper 2b	published	1,426	160	-17.4%
Paper 2b	restricted (2,183)	1,419	160	-17.0%
Paper 2a	published	1,598	178	-25.4%
Paper 2a	restricted (2,183)	1,588	176	-25.7%

Table 9: Paired race-level ROI-difference bootstrap, GBT against paper 2b and against paper 2a.

Contrast	ROI diff.	90% CI	Common races
paper 3 GBT - paper 2b exploded logit	-4.3 pp	[-13.2 pp, +4.7 pp]	2,183
paper 3 GBT - paper 2a win model	+4.4 pp	[-4.0 pp, +13.2 pp]	2,183

Both intervals contain zero. On the restricted, like-for-like universe the point estimate runs the wrong way for the GBT against both comparators — -21.3% against paper 2b’s -17.0% and paper 2a’s -25.7% — and the bootstrap intervals do not distinguish any of the three.

In the 713 of 2183 races where the GBT and paper 2b pick a different top horse, the GBT’s pick wins 20.6% of the time against paper 2b’s 22.0%, at a mean starting price of 5.60 against 5.77. A race-level bootstrap of both differences contains zero: the win-rate difference is -0.014 [-0.055, 0.024] and the price difference is -0.18 [-0.51, 0.15].

5.4 Q3 — Betting value

Single bet per race — the highest-model-win-probability qualifier under Owen’s naive rule — priced at real starting prices, win, place and each-way, the same construction as paper 2b.

Table 10: Single-bet-per-race backtest, real starting prices. 90% CI from a race-level bootstrap (B = 2000, seed 42) at the naive ratio threshold; blank where no sweep was run on that restricted comparison.

Bet type	Model	Universe	Bets	ROI	90% CI
Win	GBT	2,183	1,133	-22.3%	[-33.5%, -9.6%]
Win	Paper 2b	published	1,148	-15.1%	[-26.9%, -2.7%]
Win	Paper 2b	restricted (2,183)	1,142	-14.7%	
Place	GBT	2,183	1,133	-14.0%	[-19.5%, -8.5%]
Place	Paper 2b	published	1,145	-9.8%	[-15.5%, -4.1%]
Each-way	GBT	2,183	1,133	-13.5%	[-20.6%, -5.5%]
Each-way	Paper 2b	published	1,145	-7.7%	[-15.5%, -0.1%]

Paper 2a’s single-bet win ROI, for reference, is -25.9% published and -25.9% on the restricted universe; paper 2a reports no place or each-way figures.

The paper 3 and paper 2b intervals overlap on all three bet types.

Overall, the change of function class from a linear predictor to a gradient-boosted tree ensemble does not improve ranking, win-picking, or betting value against paper 2b, and does not narrow the gap to the market on any measure tested. Section 6 takes up what that means.

6 Discussion

The series has changed one thing at a time: paper 2a changed the feature set, paper 2b changed the objective, and this paper changes the function class. A gradient-boosted ensemble with depth-3 interactions and 698 trees ranks no better than a linear predictor with 20 terms, fitted on the identical objective, features, split and metrics as this one: the paired bootstrap against paper 2b (Section 5) contains zero on P1_rank, on Brier_place, and on test pseudo- R^2 alike. This is not two routes to the same model. Their within-race-centred scores correlate at $r = 0.918$, and they pick a different top horse in 33% of test races — two different models that rank equally well.

The reading is that the linear predictor was not the binding constraint. Neither a linear combination of these features nor a tree ensemble with real interaction depth closes any distance to the market on this objective, so whatever limits both is not the form of the function reading the features — it is more likely the information the features themselves carry. That makes the feature set the natural next lever, not the function class. It is worth stating plainly: a marginal improvement here would have left the function-class question open for a later paper to revisit; a clean tie closes it, and that is a more useful result than a marginal improvement would have been.

Set against that tie, the one contrast in the paper that is distinguishable is the market. Its discounted-Harville probabilities beat the model decisively on both ranking metrics — P1_rank by $-1.41\text{e-}03$ [$-1.55\text{e-}03$, $-1.28\text{e-}03$], Brier_place by 0.0139 [0.0126 , 0.0153] — as it has beaten every model this series has tried, on the win objective in papers 1 and 2a and on the ranking objective in paper 2b. Three changes to the model in turn have each left that gap untouched.

Five limitations are stated here rather than left for a reader to find.

- **Tuning asymmetry.** The linear models in this series were reduced by a hand-run significance rule; this model was selected from a 72-point, cross-validated grid. That is a capacity-and-selection asymmetry, and on its own a live alternative explanation for any difference found. Two things temper it: the grid is nearly flat near its maximum — only 3 of 72 points sit within one paired standard error of it — and the model given the more generous search did not win.
- **Row-level subsampling.** XGBoost draws its row subsample from individual horses; the likelihood’s own grouping unit is the race, and the two do not match. Dropping some of a race’s runners at random changes the remaining runners’ own gradients and Hessians, not merely how many rows are left. Every one of the top ten tuning configurations used subsampling at 0.7, so it clearly helps; the mismatch between the two units is unresolved.
- **Gain importance is in-sample.** It is an accounting of where the fitted loss went, not evidence a feature helps out of sample.
- **The Hessian is an approximation.** The custom objective supplies XGBoost one Hessian value per row, the diagonal of a likelihood whose true Hessian couples every runner still in contention within a race.
- **Going affinity measures ground-type form, not a preference for today’s going.** AW going is 98.9% Standard across the training races, so the feature reaching the model is mostly a horse’s record on Standard-or-Good ground relative to its own career rate. The question paper 2b deferred to this paper has been answered only in the weaker sense this data permits.

A depth-3 fit’s advantage over a depth-1 (stumps only, additive) diagnostic model appears in cross-validation — fold-mean pseudo- R^2 0.0678 against 0.0600 — and in-sample — 0.0920 against 0.0649 — where both the folds and the training data are drawn from the same era the model is fitted on. It is absent on the chronologically later test split (-0.0003 , 90% CI [-0.0016 , 0.0009]). The race-level features show the same pattern from another angle: all five permutation drops are positive on the training split (5 of 5) and 3 of 5 are negative on the test split.

We state the direction and stop there. An interval containing zero is consistent with era-specific structure that does not carry forward to later data, with ordinary sampling noise, and with a test set too small to separate the two; deciding between them needs several chronological blocks and a longer span of data than 2006–2015 provides, which we do not have. We do not adjudicate, and this observation is not offered as the paper’s explanation for the null result

above. The depth-1 fit is a diagnostic, not a candidate: its test-split numbers were computed only after model selection had closed, and it was never eligible to be chosen.

The series has now changed the features, the objective, and the function class, and none has closed the gap to the market. What is left is the information reaching the model in the first place — hand-built summaries of a horse’s run history rather than the run history itself — and learning a representation of that history directly, on the same ranking objective, is the direction under consideration next. Separately: the two models disagree on the top pick in 33% of test races, which suggests a rule that backs only where they agree. That cannot be evaluated here — the disagreement figures come from the test split, and building and testing a rule on them would be selection on the data meant to test it — and it carries a caution worth recording alongside it: two independently-specified models are more likely to agree where both are confident, which tends to mean a shorter-priced horse, and the starting-price margin bites hardest at short prices, so a higher strike rate on an agreement subset need not mean a better return. Neither is a result of this paper; both are left as open directions.

7 Appendix A: Derivation

This appendix assumes the reader is already familiar with regression trees, pruning, bagging and random forests (James et al. 2021, ch. 8) and picks up from ordinary (squared-error) boosting.

From squared error to a general loss. An ensemble built by boosting fits each new tree to the pseudo-residual of the current model: the negative gradient of the loss with respect to each observation’s current fitted value (Friedman 2001; Johansson, n.d.; Li, n.d.). Under half the squared-error loss that pseudo-residual is the ordinary residual, and the familiar fit-a-tree-to-the-residuals recipe falls out as a special case. Nothing in that recipe actually depends on the loss being squared error — only on being able to compute its gradient at the current fit — so the same procedure carries over to any differentiable loss simply by substituting its own gradient in place of the residual.

Likelihoods supply a loss. A likelihood states how probable the observed data are under a probability model; its negative logarithm is smaller exactly when the model assigns the outcome higher probability, so it is already a loss, and maximising the likelihood is minimising it. Any distributional assumption about the outcome therefore hands the boosting algorithm a loss ready to use, with no change to the algorithm beyond which gradient is computed.

The quadratic approximation and the closed-form leaf value. For most losses there is no formula for the constant a new tree’s leaf should predict, and it would have to be searched for numerically. XGBoost avoids the search by minimising a second-order (quadratic) Taylor approximation to the loss around the current fit instead, which does have a closed form. Writing g_i and h_i for the first and second derivatives of the loss at observation i ’s current fitted value,

the leaf weight that minimises the approximate loss over the observations $i \in R_m$ falling in leaf R_m (with an L_2 penalty λ on the weight, one of XGBoost’s regularisation terms) is

$$w_m^* = -\frac{\sum_{i \in R_m} g_i}{\sum_{i \in R_m} h_i + \lambda}$$

(Chen and Guestrin 2016). This is why a custom objective must supply both a gradient and a Hessian, not the gradient alone: the Hessian enters the leaf value directly, not only the choice of split.

The grouped Plackett–Luce likelihood. Every loss above — squared error, or a likelihood-based loss for a binary or count outcome — is a sum of one term per observation, so the pseudo-residual for an observation depends on that observation alone. The likelihood this paper fits does not factorise this way. Race i is the chooser and horse j the alternative; each horse carries a score z_{ij} , and win probabilities within the race are given by the softmax

$$p_{ij} = \exp(z_{ij}) / \sum_m \exp(z_{im}),$$

the sum running over the runners in race i . Writing $j_1, \dots, j_S$ for the top S finishers in order and $\mathcal{C}_{is} = \{j_s, j_{s+1}, \dots, j_J\}$ for the horses not yet ranked at stage s , the Plackett–Luce log-likelihood over a set of races is

$$\sum_i \sum_{s=1}^S \left(z_{ij_s} - \log \sum_{j \in \mathcal{C}_{is}} \exp(z_{ij}) \right),$$

and the loss is its negative. One term of this loss — the sum over s for a single race i — spans every runner still in contention in that race, not one observation in isolation; the pseudo-residual for horse j therefore depends on where every other horse in its race currently stands, and the neat one-row-at-a-time computation the earlier losses allowed no longer applies.

Gradient and Hessian. Differentiating the negative log-likelihood with respect to a single horse’s score z_{ij} gives

$$g_{ij} = \sum_{s=1}^S p_{sj} - \mathbb{1}\{j \in \text{top } S\}, \quad h_{ij} = \sum_{s=1}^S p_{sj}(1 - p_{sj}),$$

where p_{sj} is horse j ’s softmax probability at stage s (zero once j has already been placed at an earlier stage). The indicator term in g_{ij} is the direct pull toward a horse’s true finishing position; the p_{sj} terms are the softmax’s own contribution, present at every stage the horse is still a candidate.

XGBoost’s custom-objective interface takes one Hessian value per row, so it consumes h_{ij} above and nothing more. The true Hessian of this likelihood is not diagonal — differentiating g_{ij} again with respect to a different horse z_{ik} in the same race gives a further term, $-\sum_s p_{sj}p_{sk}$, that couples every pair of horses still contending at each stage — and that coupling is dropped. This is the standard approximation XGBoost makes for any grouped loss, the same one its own built-in ranking objectives rely on, not something particular to the objective built here.

8 Appendix B: Software and reproducibility

The software stack is the same as papers 1, 2a and 2b’s, described in full in paper 1’s Appendix B: R, the tidyverse, `{DBI}`/`{RMariaDB}` for the Smartform database, `{renv}` for package pinning, and Quarto for typesetting, all run as a single reproducible pipeline in which each step is cached and only rebuilt when its own inputs change. The one addition here is `{xgboost}`, used directly rather than through `{tidymodels}`/`{parsnip}`: fitting under a custom objective, a custom evaluation metric, and race-grouping information together has no `{parsnip}` interface, the same kind of exception papers 1 and 2 make for `{mlogit}`.

One part of the pipeline is expensive enough, and consequential enough for §4’s tuning-asymmetry disclosure, to document in its own right: the grid search behind the selected configuration. The full factorial grid, before any reduction, also varies `colsample_bytree` over two values — 144 points under the same five-fold cross-validation, projected at roughly 12.9 hours, above this project’s working limit of about 12 hours for a single run. Fixing `colsample_bytree` at its lower value alone cut the grid to the 72 points actually run, projected at roughly 6.5 hours; that was enough, and the further rungs available (also fixing `subsample`, dropping the lowest `eta`, or narrowing `min_child_weight`) were not needed. The measured cost came in higher than projected: cross-validation summed to 10.9 hours across the 72 points, meaning the un-reduced grid would have taken roughly twice as long — the cut was necessary, not a precaution that turned out unneeded. Most of that time is spent evaluating the custom objective and its matching evaluation metric, once each per boosting round: every round needs the gradient/Hessian pair for the next tree and the held-out score for early stopping, and this dominates the per-round cost, with building the trees themselves comparatively cheap.

Two further facts about the search are recorded here because they matter for anyone re-running or auditing it, rather than for the argument of the paper itself. No grid point diverged: a guard logs, rather than silently discards, any configuration whose scores run away during boosting, and it recorded zero such events across all 72 points. And every one of the top ten points by fold-mean score uses `subsample = 0.7` — not itself part of the paper’s argument, but worth recording alongside the selected configuration for anyone tuning a similar grid on this objective.

Acknowledgments

I have been interested in modelling horse racing since first acquiring the Smartform database more than a decade ago. The project has been picked up and put down many times, the obstacle almost always being the gap between an idea and a running implementation. What is different now is that working alongside Claude Code (Anthropic’s coding agent), with web-based Claude as a research companion, has compressed that gap dramatically. I am grateful to my brother, Michael Gillen, for the suggestion to build the model on All-Weather races — the foundation of the scope choices set out in Section 2. The design choices — dataset, scope, model, comparison,

editorial line — are mine. The implementation, the bulk of the draft prose, and a great deal of the verification were Claude Code’s, under my direction; every output was reviewed and edited before it reached the paper. I do not yet know what general lessons this experiment points to about the value of human–AI collaboration in applied statistics. That uncertainty is part of why I am doing the work, and these papers are partly a record of that experiment.

References

- Chen, Tianqi, and Carlos Guestrin. 2016. “XGBoost: A Scalable Tree Boosting System.” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD ’16)* (San Francisco, CA), 785–94.
- Friedman, Jerome H. 2001. “Greedy Function Approximation: A Gradient Boosting Machine.” *The Annals of Statistics* 29 (5): 1189–232.
- James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2021. *An Introduction to Statistical Learning with Applications in R*. 2nd ed. Springer.
- Johansson, Richard. n.d. *An Intuitive Explanation of Gradient Boosting*.
- Li, Cheng. n.d. *A Gentle Introduction to Gradient Boosting*.